Proyecto 2 - Documento de diseño

Cambios respecto a la entrega anterior:

- ClubSeneca: se crea getEquipos(): devuelve List<Equipo> porque la consola necesita listar equipos (HU-SOC-02, HU-ENT-02)
- ClubSeneca: se crea getSolicitudesPendientes (): devuelve las solicitudes en estado PENDIENTE porque El administrador debe elegir cuál revisar (HU-ADM-02)
- ClubSeneca: se crea getEmpleados(): devuelve List<Empleado> porque Se usa en resolverSolicitud para validar el mínimo de personal (HU-ADM-02)
- ClubSeneca: inscribirJugador(..): devuelve void y lanza ReglaNegocioException porque debe informar el motivo del rechazo (HU-SOC-02, CA-2, CA-3, CA-4)
- ClubSeneca: resolverSolicitud(..): lanza ReglaNegocioException cuando la aprobación dejaría menos de 2 entrenadores o 1 fisioterapeuta (CA-2), o cuando la solicitud no existe o ya fue resuelta; en esos casos la solicitud no cambia de estado (antes se rechazaba automáticamente).
- ClubSeneca: se crea registrarUsuario(Usuario): devuelve boolean agrega el usuario si el login no está repetido y, si es Administrador, le asigna el club. Se crea para crear los datos iniciales para poder iniciar sesión.
- ClubSeneca: se modifico darDeBajaSocio() para que no fallara con socios sin equipo.
- Equipo: agregarJugador(..): antes devolvía boolean, ahora devuelve void y lanza ReglaNegocioException porque debe informar el motivo del rechazo (HU-SOC-02, CA-2, CA-3, CA-4)
- Administrador: resolverSolicitudTurno(..): se agrega throws ReglaNegocioException que es necesario por el punto 5 de esta lista
- Empleado: Se modificó toJSON() para añadir el tipo “Empleado” y se agrego el metodo fromJSON()

Las historias de usuario que escogimos para esta entrega:

1. **HU-SOC-02**
2. **HU-ENT-02**
3. **HU-ADM-02**

Cambios introducidos para soportar los nuevos requerimientos y con la implementación de la interfaz:

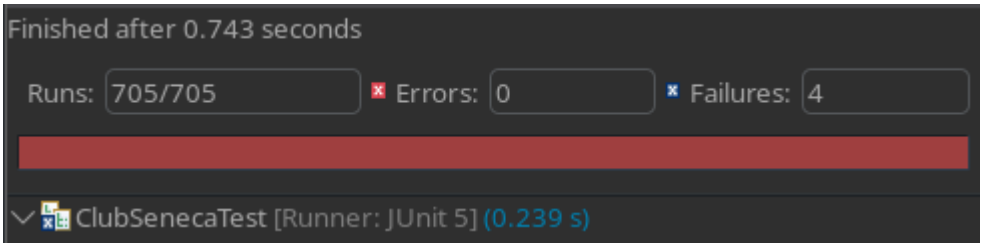
- Clases nuevas:
    - el paquete TorneosInternos
    - NucleoSistema.ClasificacionPreferencial
    - Tienda_Cafeteria_Inventario.BonoDescuento
    - el paquete Consola
    - ParticipanteExterno

    - Consola.OpcionRegistrarParticipanteExterno
    - Consola.OpcionCorregirGanadorTorneo

- ClubSeneca: la lista torneos y los métodos getTorneos, agregarTorneo, inscribirEnTorneo, retirarJugadorDeTorneo, retirarEquipoDeTorneo, iniciarTorneo, registrarGanadorTorneo, esElegiblePreferencial, getDisciplinas, registrarDisciplina, obtenerDeporte, buscarSocio, getProductosDisponibles y registrarVentaConBono. También guarda y carga los torneos en JSON. Además crea registrar ParticipanteExterno que si el nombre se encuentra vacío, la edad no se encuentra entre 5 y 100, si el login ya está ocupado o si no se indica el nivel de ninguna disciplina lanza una regla de negocio. Así mismo se crea un existeLogin para que la consola avise de un login ocupado antes de pedir el resto de los datos del usuario, adicionalmente buscarJugadorTorneo(login) para que el compañero de dobles pueda ser socio, empleado o participante externo.
- Usuarios reconoce el tipo “ParticipanteExterno” dentro del JSON
- Socio: ahora implementa JugadorTorneo, tiene bonos (getBonos, agregarBono, que se guardan en el JSON) y el método esEmpleado().
- Empleado: esEmpleado() y tieneTurnoEl(fecha), para la regla de que un empleado de turno no se inscribe.
- Compra con bono: Tienda.getArticulos(), CafeteriaSocios.getProductos() y Venta.aplicarBono().
- module-info: para las pruebas.
- PremiacionCompetitiva el constructor lanza ReglaNegocioException si la tarifa es menor o igual a 0.
- ParticipanteExternos: se crea con el fin de que estudiantes de la universidad y personas externas al club y a la institución participen en torneos individuales. Esta hereda de usuario e implementa la interfaz de JugadorTorneo. Esta clase guarda la edad, si es estudiante, el nivel por disciplina, los bonos y son guardados en el JSON. Al no ser socio, no entra a equipos y tampoco compra en la tienda

Test del Proyecto:

Se realizaron un total de 705 pruebas al código entregado en el Proyecto 1, estas pruebas lograron una cobertura superior al 95% del código trabajado. Siguiendo los resultados estas pruebas se encontraron los siguientes errores:



Paquete NucleoSistema, Clase ClubSeneca:

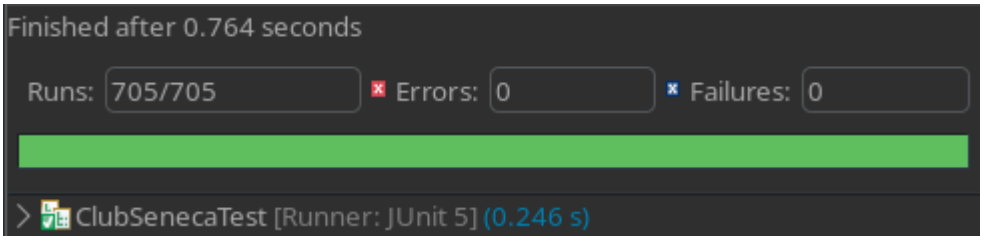
- Falla en testDarDeBajaSocioDosVeces()
- Falla en testDarDeBajaSocioSinEquipos()

- Falla en testDarDeBajaSocioSinEquiposLimpiaSusDatosIndividuales()

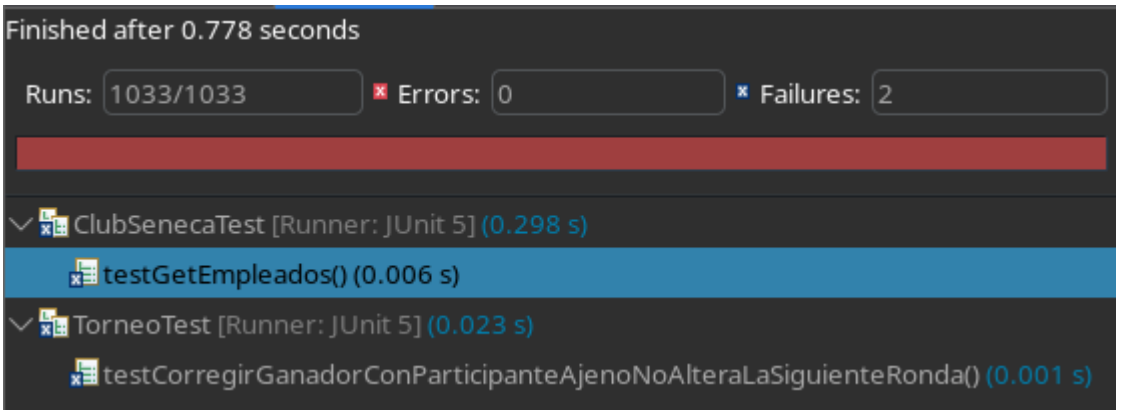
Paquete Usuarios, Clase Administrador:

- Falla en testDarDeBajaSocioSinEquipos()

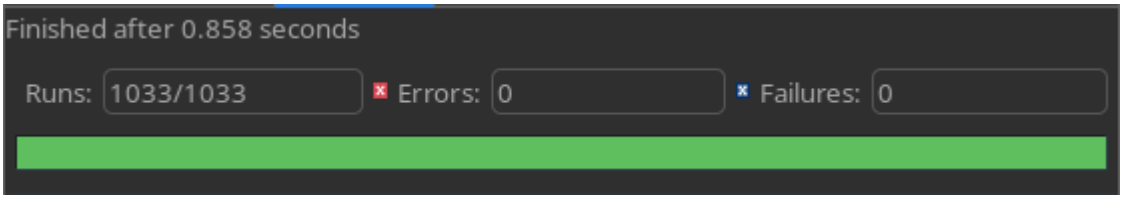
Mediante estos errores se identificó la falla en el método darDeBajaSocio que fallaba con socios sin equipos por lo que se corrigió y se obtuvo que el proyecto corría sin errores ni fallos.



Posterior al testeo del código del Proyecto 1, se crearon los nuevos test que cubren las implementaciones realizadas en el Proyecto 2. Estos incluyen un total de 328 nuevos tests distribuidos entre ClubSececaTest, TorneosInternos_Tests y ClasificacionPreferencialTest. Mediante estos nuevos test se identificaron las siguientes fallas en el programa realizado en el Proyecto 2:



Finalmente se corrigió el código del proyecto siguiendo los errores dados por los test, pues Empleados no se guardaba correctamente en el JSON si no era ni Entrenador ni Fisioterapeuta por lo que se creó el método Empleado.fromJSON y se modificó Empleado.toJSON, Torneo fallaba por que al corregir el ganador, no se verifica si el nuevo ganador era un ganador válido.





Para el caso de las pruebas respecto a las historias de usuario iniciales fueron exitosas empezando porque una de las tres ya se encontraba incorporada, las funcionalidades adicionales también resultaron exitosas por lo cambios realizados para la implementación de estas. Respecto a las otras pruebas se realizó la implementación con paciencia y bastante tiempo con la finalidad de hacer que pasaran las pruebas que se tenían planeadas para esta, se corrieron los tests y después de realizar ciertos cambios y ajustes resultaron exitosas cumpliendo la funcionalidad que se tenían propuestas inicialmente y a lo largo del proyecto.

